

시스템 해킹(pwnable)

포너블이란? & 세션소개
(OT)

포너블(pwnable) 개요

시스템 해킹이란 무엇인가...

시스템 해킹이라며...

- 시스템 해킹 : 프로그램이나 시스템의 취약점을 분석하고, 이를 이용해 권한을 상승시키거나 임의의 코드를 실행하는 일련의 과정

```
시스템 해킹
├─ 운영체제 취약점 분석 (권한 상승, 커널 익스플로잇)
├─ 네트워크 기반 공격 (패킷 분석, 서비스 공격)
├─ 소프트웨어 취약점 공격
│   └─ 리버스 엔지니어링
│       └─ **포너블(Pwnable)**
│           ├── 메모리 구조 이해
│           ├── 버퍼 오버플로우(BOF)
│           ├── 포맷 스트링 버그
│           └─ 기타 메모리 취약점
└─ 기타 시스템 내부 공격 (디버깅, 후킹, 인젝션 등)
```

너무 넓다!

→ 포너블에 집중

◀ 지피티 피셜 시스템 해킹 분야

포너블이란?

- “시스템 해킹 분야의 한 갈래로, 프로그램의 메모리 취약점을 이용해 시스템을 장악하거나 동작을 조작하는 기술”
- 주로 운영체제나 프로그램의 보안 취약점을 분석하고 공격하여, 관리자 권한을 획득하거나 프로그램의 흐름을 해커가 원하는 방식으로 바꾸는 것이 목적
- CTF의 문제 유형

왜 중요한가...

1. 해킹 체험

- 시스템을 어떻게 뚫을 수 있는지를 배우는 분야.

2. 실전 보안에 꼭 필요한 역량

- 포너블은 취약점 분석과 침투 테스트에 쓰이는 기술.

3. CTF 대회에 유리한 스킬셋

- 해킹 대회(CTF)에서 포너블 문제는 항상 출제됨.

4. 시스템 내부를 꿰뚫는 사고력 향상

- 메모리 구조, 함수 호출, 스택 프레임 등 컴퓨터 구조를 이해하게 됨.

세션소개

이게 뭐하는 강의일까

세션소개

- 소개 : EVI\$ION의 YB들을 위한 시스템 해킹 입문 강의
(발표자도 입문자!)
- 목표 : 기초부터 차근차근 시스템 해킹 공부 시작하기

커리큘럼

- 1회차 : 시스템 메모리 구조 & 함수 호출 원리
→ 우리가 프로그램을 실행할 때, 컴퓨터 메모리는 어떻게 될까?
- 2회차 : 어셈블리, gdb, 셸
→ 컴퓨터는 어떤 방식으로 소스코드를 이해하고 실행할까?
- 3회차 : BOF 개념 & pwntools
→ 어떻게 메모리를 조작해서 프로그램 흐름을 바꿀 수 있을까?

진행 방식

- 이론 + 실습 + 과제
- 매 주차마다 복습 목적 과제가 나가요
 - 과제에는 GPT/인터넷 검색 허용(적극 추천)
 - 제출은 다음 세션 시작 전(18:30) 까지, 늦제출은 금요일(23:59)까지
- 어려워요 → 저도요...
 - 동아리 시간이 끝나고 저는 30분 동안 남아 있어요.
 - 내용 질문, 과제 질문 다 받아요.
 - “이거 너무 기초적인 질문인데...” → 대환영

시스템 메모리 구조 & 함수 호출 원리

우리가 프로그램을 실행할 때, 컴퓨터 메모리는 어떻게 될까?

목차

- 프로그램 메모리 구조 4영역
- 레지스터 개념
- 함수 호출 시 스택 변화
- 과제 안내

프로그램 메모리 구조 4영역

프로그램이 컴퓨터 메모리에 어떻게 저장되어 있는가

```
1    #include <stdio.h>
2
3    int sum(int x, int y) {
4        return x + y;
5    }
6
7    int main() {
8        int a = 2, b = 3, c;
9        c = sum(a, b);
10       printf("meow ^._.^");
11   }
```

귀여운 C 소스코드

이게 메모리에 어떻게 적재될까?

※ 참고 ※

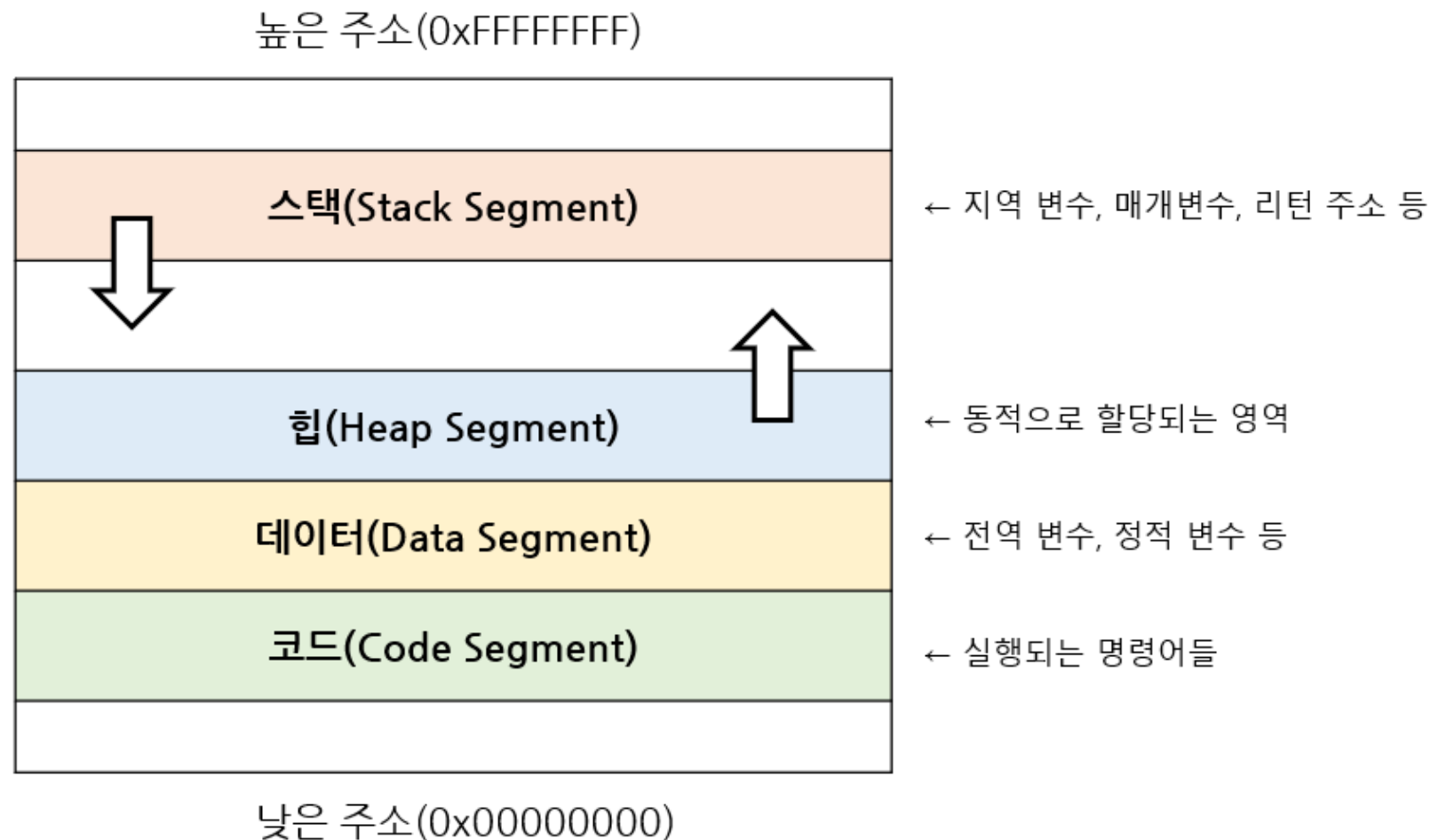
지금 말하는 메모리는 RAM

저장소(디스크) 말하는 거 아님!

프로그램 메모리 구조 4영역

- 코드 : 실행 코드가 들어있는 공간
- 데이터 : 초기화된 데이터가 저장되는 공간
- 힙 : 프로그램 실행 중 동적으로 사용되는 공간
- 스택 : 함수 호출 시 인자, 지역 변수, 리턴 주소 등이 저장되는 공간

프로그램 메모리 구조 4영역



코드(Code)

- 코드 세그먼트(Code Segment) / 텍스트 세그먼트(Text Segment)
- 시스템이 알아들을 수 있는 실행 명령어(기계어)가 저장되어 있다
- 컴파일된 함수 코드와 제어 흐름 관련 명령어들 포함
- 실행과 읽기 권한만 부여

데이터(Data)

- 프로그램 실행 시 사용되는 데이터가 저장됨.
- 전역 변수(global variable)와 정적 변수(static variable)들이 들어감.
- 세분화한 구조
 - 데이터 세그먼트(.data) : 초기화 O 전역 변수, 정적 변수.
 - BSS 세그먼트(.bss) : 초기화 X 전역 변수, 정적 변수.
 - 리소스 데이터 세그먼트(.rodata) : 초기화 O 읽기 전용. (주로 상수)

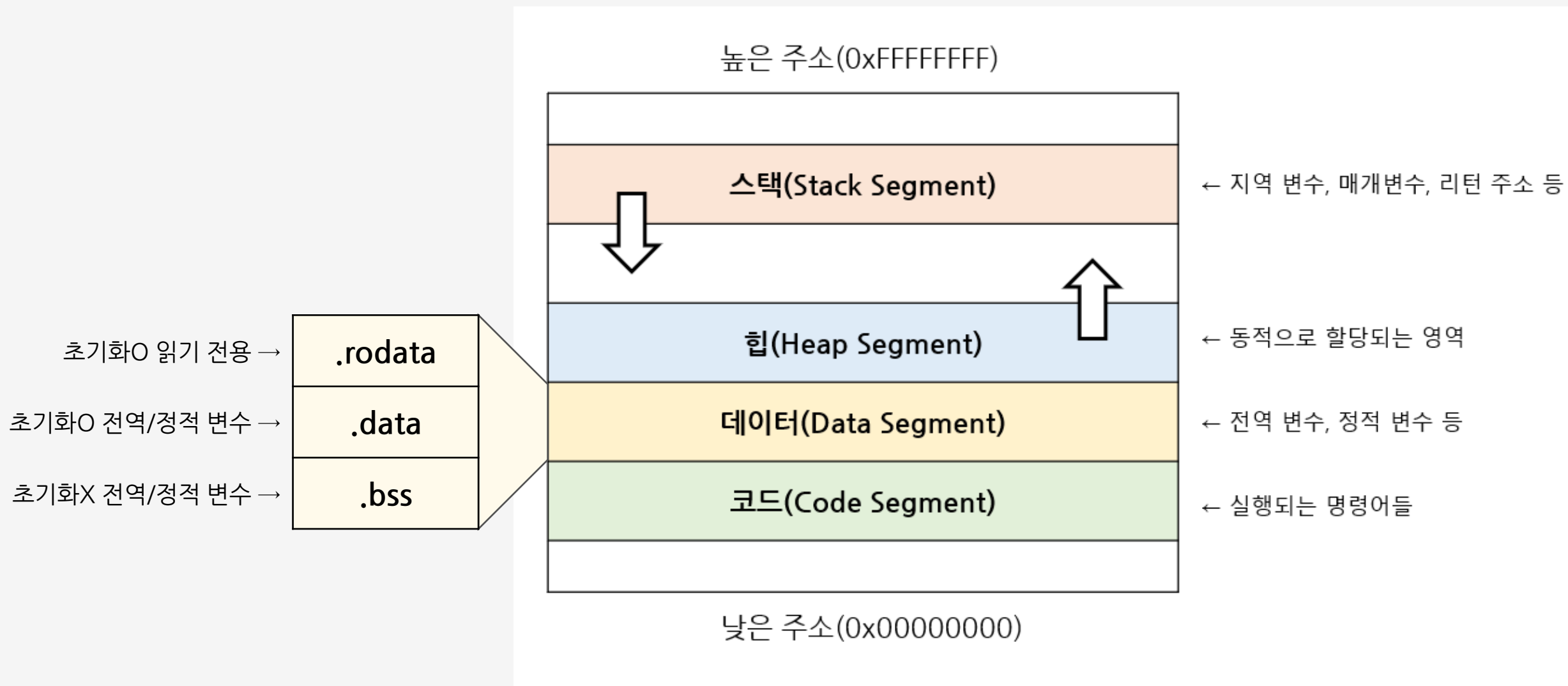
힙(Heap)

- 프로그램이 실행 중에 동적으로 사용되는 영역
- 하위 메모리 주소에서 상위 메모리 주소로 확장
- malloc, calloc, new 등에 의해 실행 중 동적으로 할당
- 개발자가 직접 free() 또는 delete()로 해제해야 함
- 메모리를 해제하지 않고 반복적인 메모리 할당이 누적되면 메모리 누수 (Memory Leak) 발생

스택(Stack)

- 프로그램 로직이 동작하기 위한 인자와 프로세스 상태가 저장되는 영역
- 함수 호출 시 지역 변수, 매개변수, 반환 주소 등이 저장
- 상위 메모리 주소에서 하위 메모리 주소로 확장

프로그램 메모리 구조 4영역



```

1  #include<stdio.h>
2  #include<stdlib.h>
3
4  int global1 = 1;
5  int global2;
6
7  int sum(int x, int y) {
8      static int call_count = 0
9      call_count++;
10     return x + y;
11 }
12
13 int main(){
14     const int size = 10;
15     int* local_ptr;
16     local_ptr = (int*)malloc(size*sizeof(int));
17     free(local_ptr);
18
19     int a = 2, b = 3, c;
20     c = sum(a, b);
21     printf("meow ^._.^");
22
23     return 0;
24 }

```

퀴즈1

다음 중 스택 영역에 저장되는 것을 모두 고르세요.

1. stdio.h, stdlib.h
2. global1
3. global2
4. x, y
5. call_count
6. size
7. local_ptr
8. a, b, c
9. "meow ^._.^"

```

1  #include<stdio.h> //헤더파일은 메모리에 없음. 전처리에 의해 치환.
2  #include<stdlib.h>
3
4  int global1 = 1; //초기화된 전역변수 : .data 세그먼트
5  int global2; //초기화 안된 전역변수 : .bss
6
7  int sum(int x, int y) { //x, y는 함수 인자 : 스택
8      static int call_count = 0; //정적 변수 : .data 세그먼트
9      call_count++;
10     return x + y;
11 }
12
13 int main(){
14     const int size = 10; //지역 const 변수 : 스택
15     int* local_ptr; //지역 포인터 변수 : 스택
16     local_ptr = (int*)malloc(size*sizeof(int)); //malloc으로 할당한 메모리 : 힙
17     free(local_ptr);
18
19     int a = 2, b = 3, c; //지역변수 a, b, c : 스택
20     c = sum(a, b);
21     printf("meow ^._.^"); //"meow ^._.^"는 문자열 상수 : .rodata
22
23     return 0;
24 }

```

정답!

다음 중 스택 영역에 저장되는 것을 모두 고르세요.

1. stdio.h, stdlib.h
2. global1
3. global2
4. x, y
5. call_count
6. size
7. local_ptr
8. a, b, c
9. "meow ^._.^"

레지스터

컴퓨터가 어디를 보고 있고, 무엇을 계산하고 있지 알려주는 표지판

레지스터란?

- CPU 내부의 연산용 임시 저장소
- CPU 연산과 어셈블리어의 동작에 사용된다
- 범용 레지스터, 세그먼트 레지스터, 포인터 레지스터, 인덱스 레지스터, 플래그 레지스터 등으로 구분
- 각각의 레지스터는 특정 연산이나 메모리 접근에 특화

레지스터 종류

- 범용 레지스터(General-Purpose register)
 - 데이터 저장, 연산, 주소 계산 등 다목적으로 사용되는 레지스터
- 세그먼트 레지스터(Segment register)
 - 코드 세그먼트, 데이터 세그먼트, 스택 세그먼트를 가리키는 주소(base 주소)가 들어 있는 레지스터
- 포인터 레지스터(Pointer register)
 - 원래 범용 레지스터에 속하지만, 스택 프레임, 스택 최상단, 명령어 포인터 등 주소를 가리키는 특별한 용도로 자주 쓰임

레지스터 종류

- 인덱스 레지스터(Index register)
 - 배열, 문자열 처리 등에서 인덱스로 사용하는 레지스터
- 플래그 레지스터(Flag register)
 - 연산 결과 및 시스템 상태와 관련된 여러 가지 플래그 값들이 저장됨

[참고] 레지스터 종류

범용 레지스터

레지스터	비트	설명
EAX (Accumulator)	32	연산 결과, 함수 리턴 값 저장.
EBX (Base)	32	특정 주소 저장. 주소 계산을 위해 사용.
ECX (Counter)	32	루프 반복 횟수나 시프트 비트 수 기억.
EDX (Data)	32	일반 데이터 저장. 곱셈/나눗셈 시 보조.

세그먼트 레지스터

레지스터	비트	설명
CS (Code Segment)	16	실행할 코드가 있는 세그먼트.
DS (Data Segment)	16	일반 데이터가 있는 세그먼트.
SS (Stack Segment)	16	스택 영역의 베이스 주소.
ES, FS, GS	16	여분의 레지스터.

인덱스 레지스터

레지스터	비트	설명
EDI (Destination Index)	32	목적지 인덱스. 목적지 주소의 값 저장.
ESI (Source Index)	32	출발지 인덱스. 출발지 주소의 값.

포인터 레지스터

레지스터	비트	설명
EBP (Base Pointer)	32	베이스 포인터. 함수 호출 시 스택 프레임의 기준점.
ESP (Stack Pointer)	32	스택 포인터. 스택 최상단(top)을 가리킴.
EIP (Instruction Pointer)	32	명령 포인터. 다음에 실행할 명령어의 주소.

플래그 레지스터

분류	레지스터	설명
상태 플래그	CF (Carry Flag)	연산 수행에서 carry 혹은 borrow 발생.
	PF (Parity Flag)	최하위 비트가 0인 경우.
	AF (Adjust Flag)	산술 연산에서 BCD 숫자 캐리 발생.
	ZF (Zero Flag)	연산 결과가 0인 경우.
	SF (Sign Flag)	연산 결과가 음수인 경우.
	OF (Overflow Flag)	산술 연산 결과가 표현 가능 범위 초과.
제어 플래그	IF (Interrupt Enable Flag)	인터럽트가 사용가능한 경우
	DF (Direction Flag)	설정 시 문자열 연산에서 역순으로 읽음.
시스템 플래그	VM (Virtual 8086 Mode)	Virtual 8086 모드 활성화 여부.
	RF (Resume Flag)	디버깅 시 예외 재발 방지.
	AC (Alignment Check)	메모리 정렬 검사

이 친구들을 알아야 해요

- 스택 포인터 : ESP/RSP – 스택의 최상단
- 베이스 포인터 : EBP/RBP – 함수의 기준점
- 명령어 포인터 : EIP/RIP – 지금 실행 중인 명령어 위치
- 연산 결과용(관습적) : EAX/RAX – 연산 결과/리턴값



* R- : 64bit 아키텍처

* E- : 32bit 아키텍처

함수 호출 시 스택 변화

프로그램에서 함수를 호출했을 때 스택에서 일어나는 일

우리가 함수를 호출할 때...

```
1    #include <stdio.h>
2
3    int sum(int x, int y) {
4        return x + y;
5    }
6
7    int main() {
8        int a = 2, b = 3, c;
9        c = sum(a, b);
10       printf("meow ^._.^");
11    }
```

설명하기 편하게 야옹이는 빼기로...

프로그램을 실행할 때 스택 구조는
어떻게 될까?

함수 호출 규약

- 함수 호출 시 인자 전달, 리턴 값 처리, 스택 정리 방식을 정의하는 규칙
- x86 32bit architecture cdecl calling convention 기준
 - 인자를 오른쪽에서 왼쪽으로 스택에 push
 - 리턴 값은 EAX 레지스터에 저장됨
 - 인자는 호출자(caller)가 정리

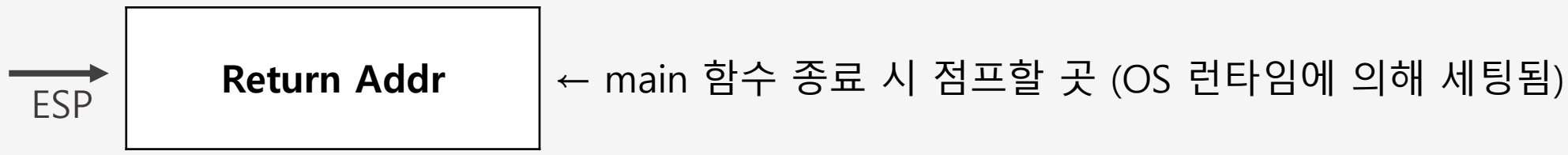
[ebp+12]	→ 두 번째 인자
[ebp+8]	→ 첫 번째 인자
[ebp+4]	→ 리턴 주소
[ebp]	→ 이전 EBP (base pointer)
[ebp-4]	→ 지역 변수
[esp]	→ 현재 스택 포인터

▲ 스택 프레임 구조

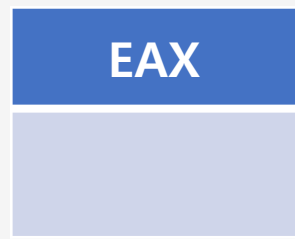
main() 함수

esp → return addr

높은 주소

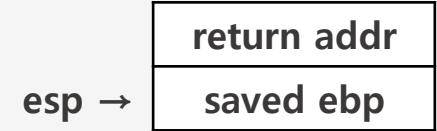


낮은 주소

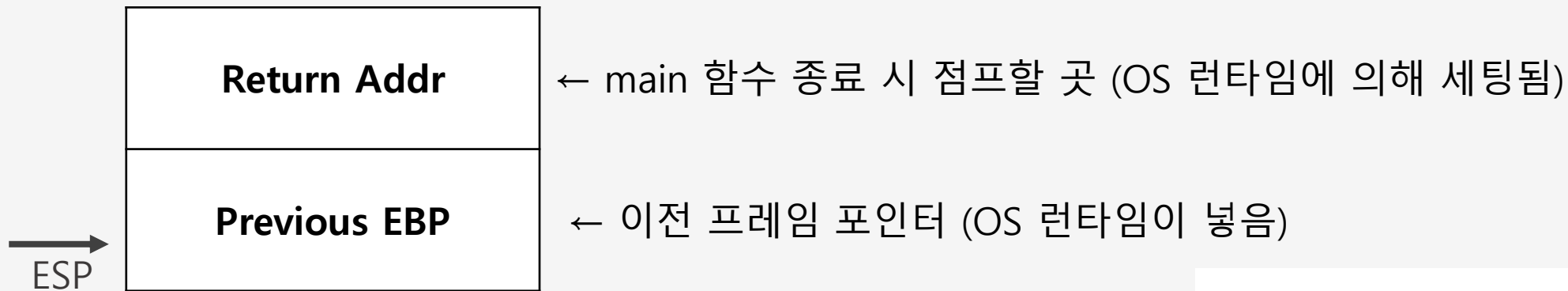


```
1      #include <stdio.h>
2
3      int sum(int x, int y) {
4          return x + y;
5      }
6
7      int main() {
8          int a = 2, b = 3, c;
9          c = sum(a, b);
10         printf("meow ^._.^");
11     }
```

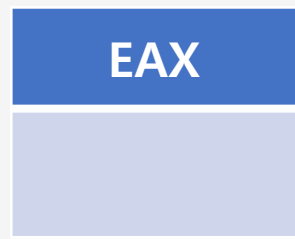

main() 함수



높은 주소



낮은 주소

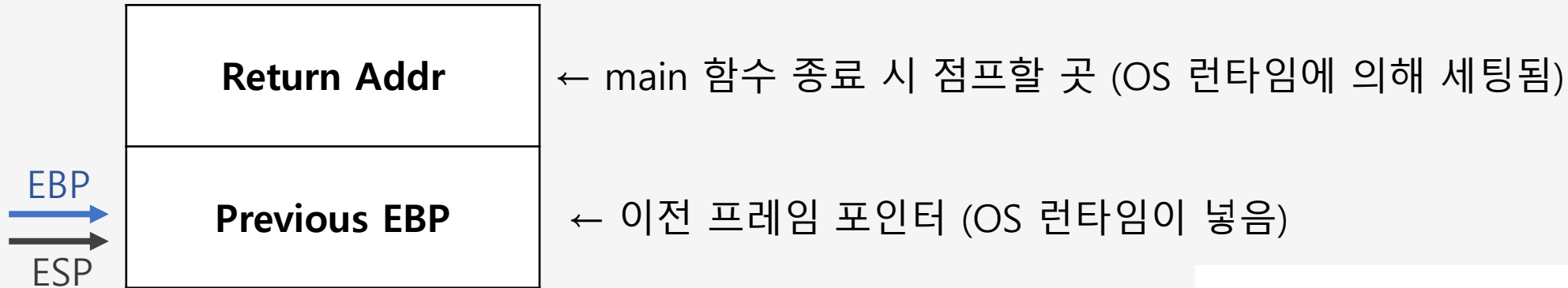


```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

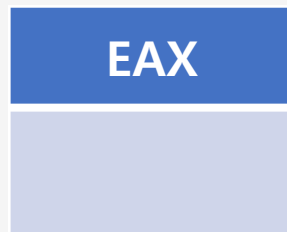
main() 함수

ebp, esp →	return addr
	saved ebp

높은 주소



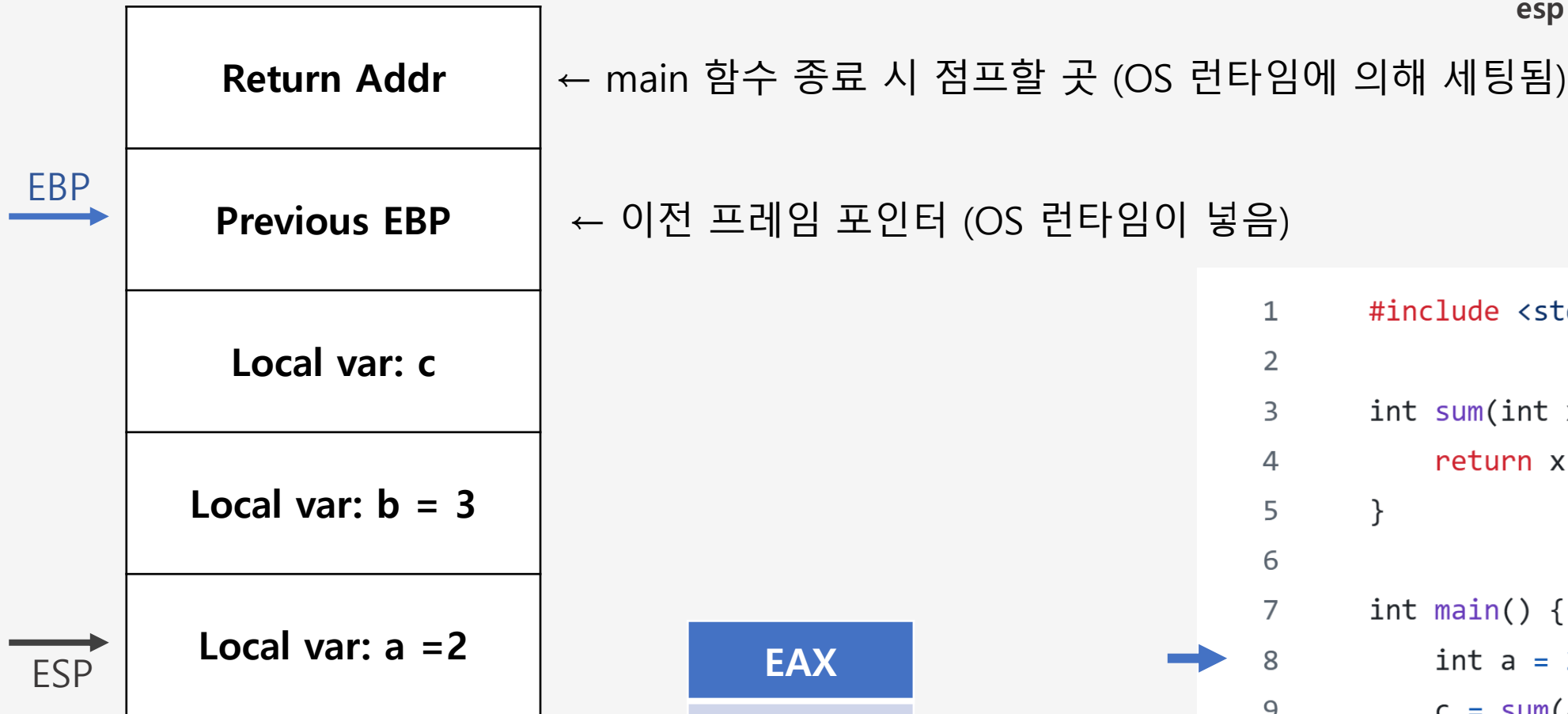
낮은 주소



```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

main() 함수

높은 주소



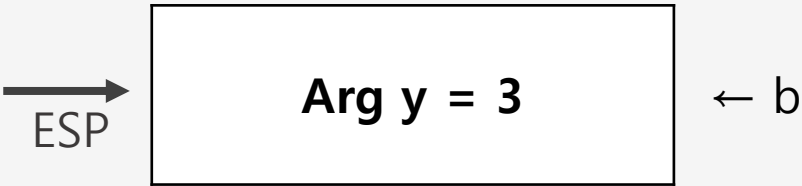
낮은 주소

ebp →	return addr
	saved ebp
	local: c
	local: b
esp →	local: a

```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 함수 호출

높은 주소



	return addr
ebp →	saved ebp
	local: c
	local: b
	local: a
esp →	arg: y

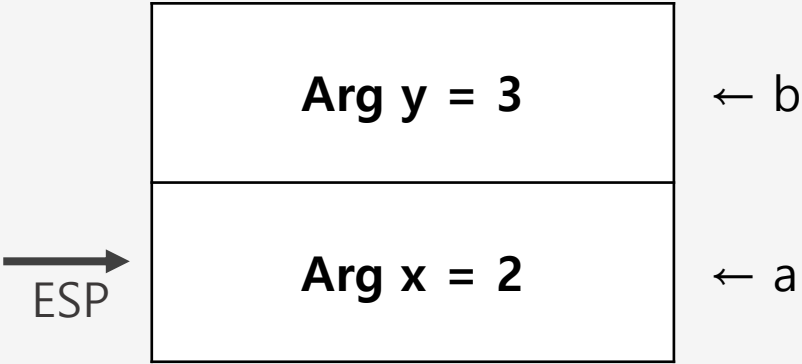
낮은 주소



```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 함수 호출

높은 주소



낮은 주소

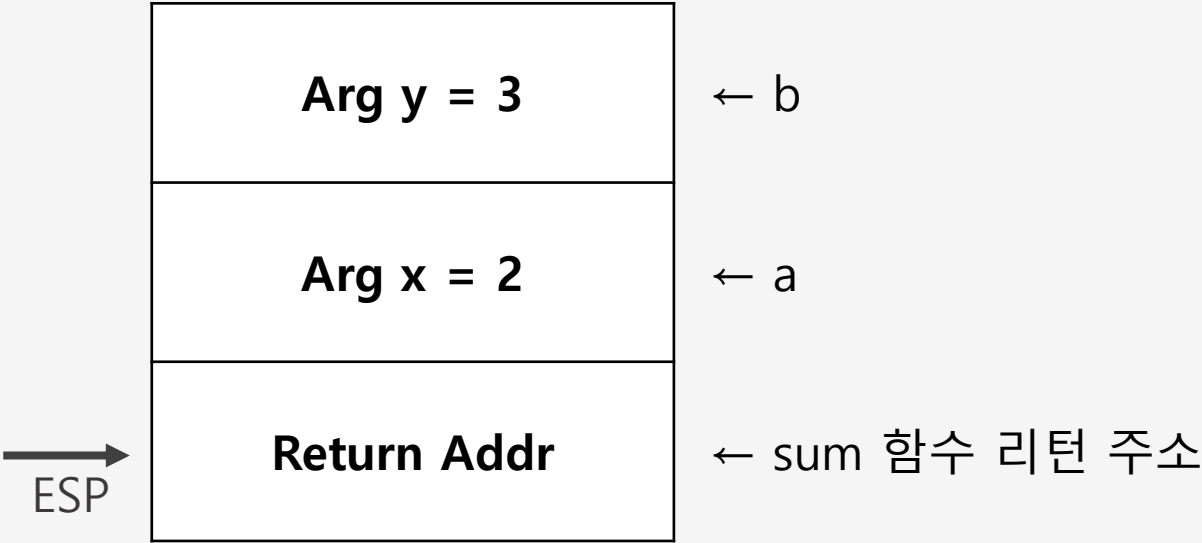


	return addr
ebp →	saved ebp
	local: c
	local: b
	local: a
	arg: y
esp →	arg: x

```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 함수 호출 후

높은 주소



낮은 주소

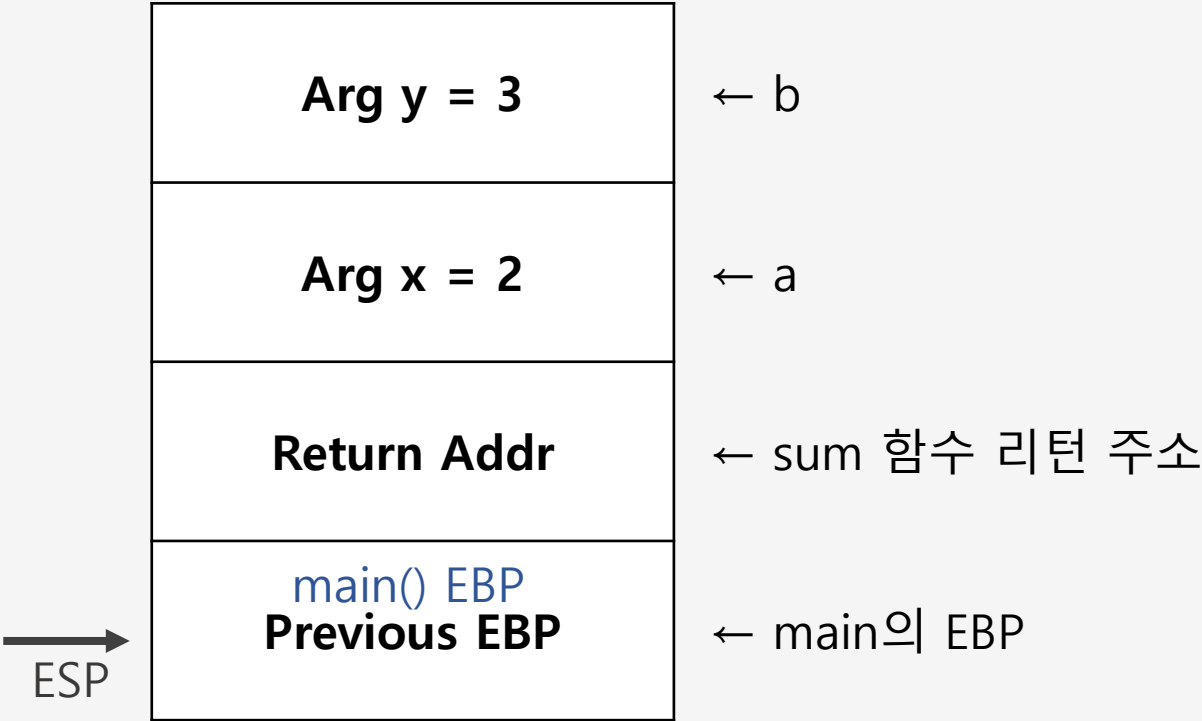


	return addr
ebp →	saved ebp
	local: c
	local: b
	local: a
	arg: y
	arg: x
esp →	return addr

```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 함수 호출 후

높은 주소



낮은 주소

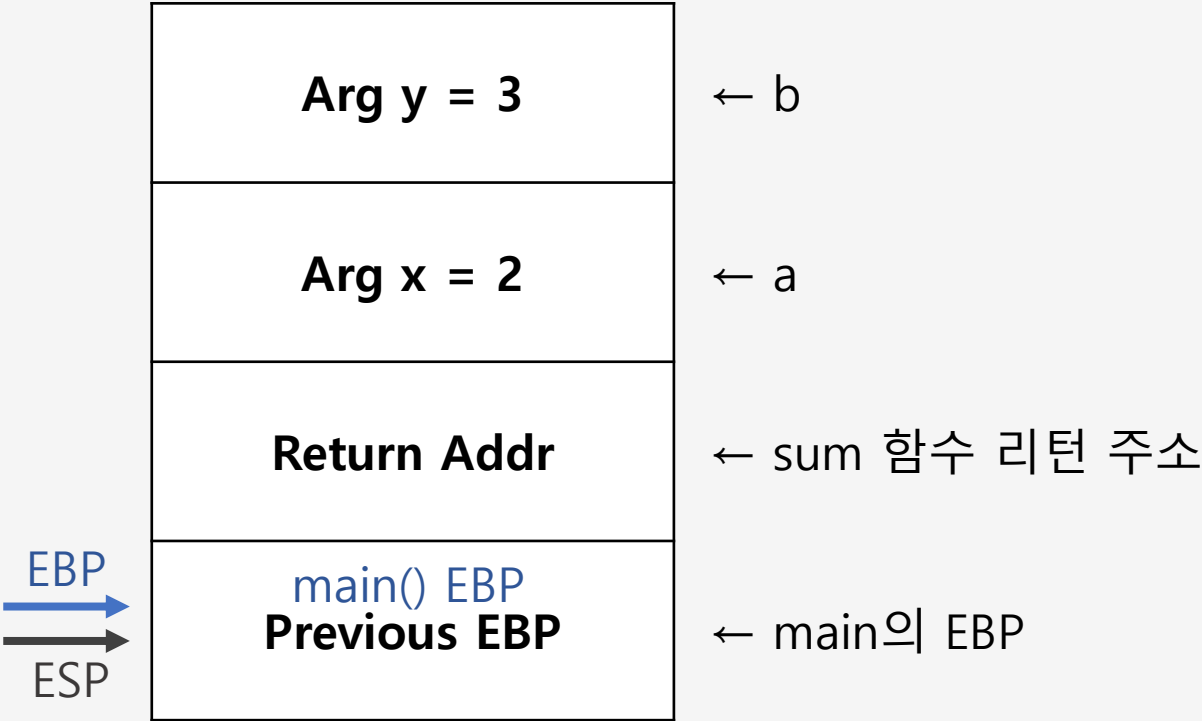


ebp→	return addr
	saved ebp
	local: c
	local: b
	local: a
esp →	arg: y
	arg: x
	return addr
	saved ebp

```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 함수 호출 후

높은 주소



낮은 주소



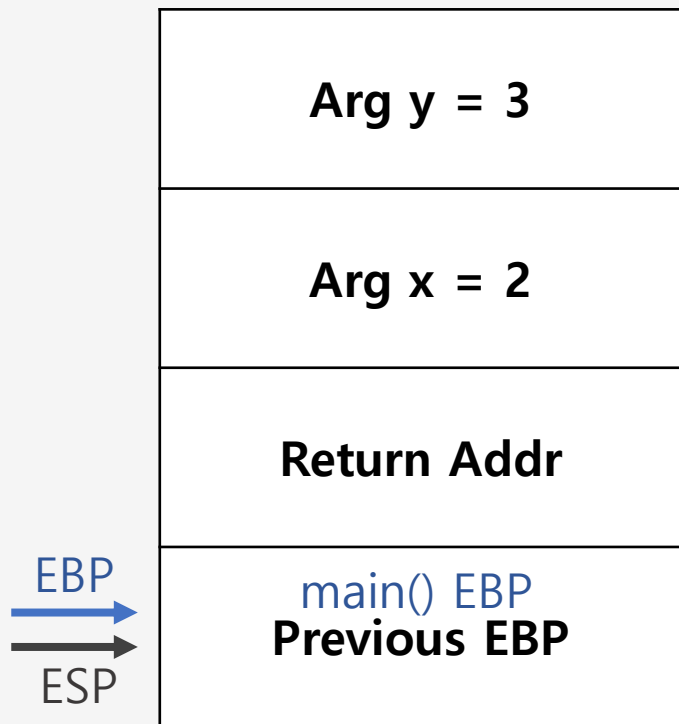
return addr
saved ebp
local: c
local: b
local: a
arg: y
arg: x
return addr
saved ebp

ebp, esp →

```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```


sum(a, b) 실행 후 리턴 직전 (ret 직전)

높은 주소



← ret 이 주소로 점프 예정

낮은 주소



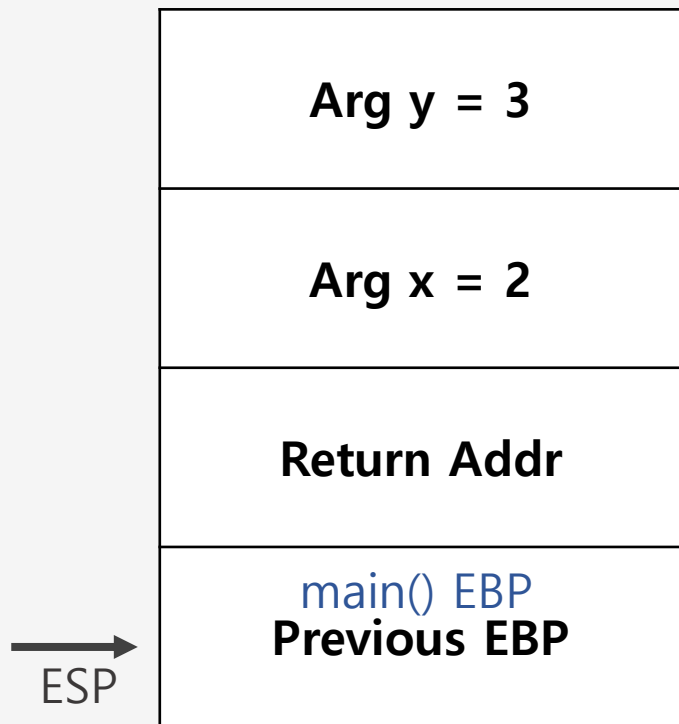
return addr
saved ebp
local: c
local: b
local: a
arg: y
arg: x
return addr
saved ebp

ebp, esp →

```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 실행 후 스택 정리

높은 주소



← ret 이 주소로 점프 예정

낮은 주소



ebp→

esp →

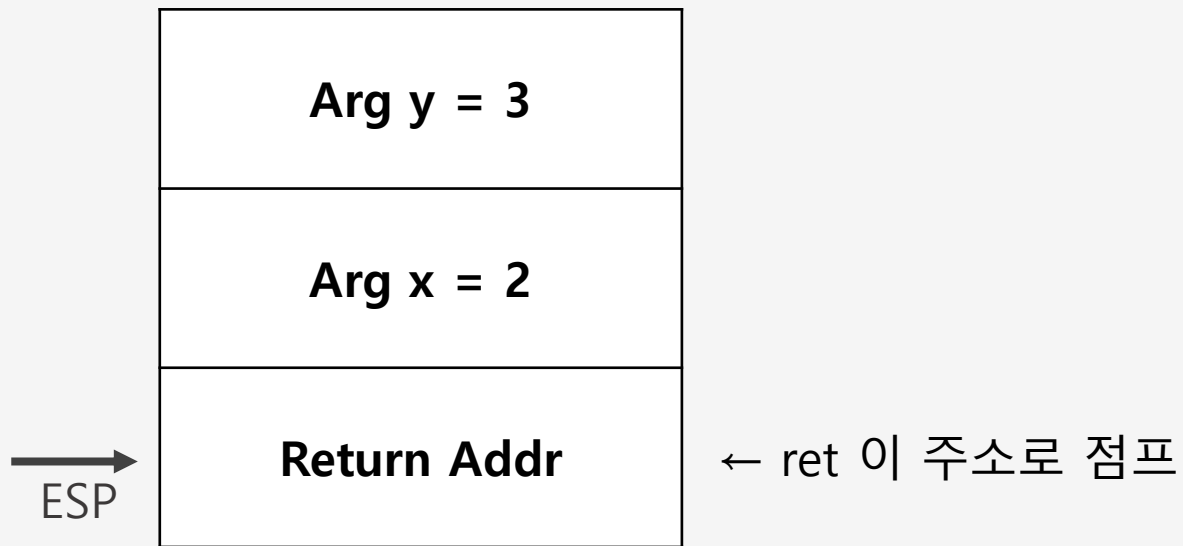
return addr
saved ebp
local: c
local: b
local: a
arg: y
arg: x
return addr
saved ebp



```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 실행 후 스택 정리

높은 주소



낮은 주소

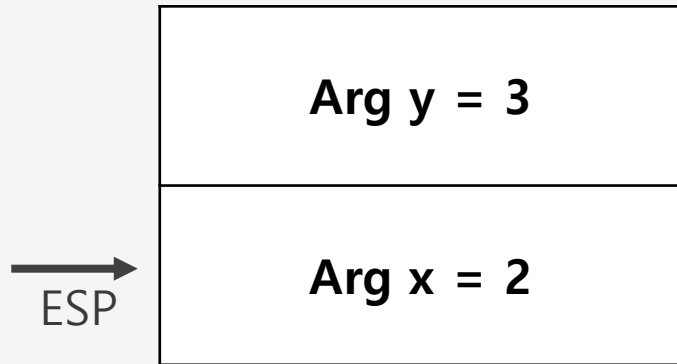


	return addr
ebp →	saved ebp
	local: c
	local: b
	local: a
	arg: y
	arg: x
esp →	return addr

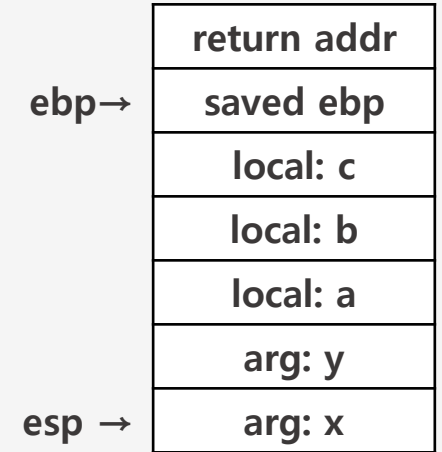
```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum(a, b) 실행 후 스택 정리

높은 주소



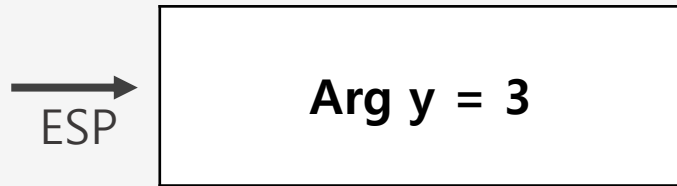
낮은 주소



```
1    #include <stdio.h>
2
3    int sum(int x, int y) {
4        return x + y;
5    }
6
7    int main() {
8        int a = 2, b = 3, c;
9        c = sum(a, b);
10       printf("meow ^._.^");
11    }
```

sum(a, b) 실행 후 스택 정리

높은 주소



	return addr
ebp →	saved ebp
	local: c
	local: b
	local: a
esp →	arg: y

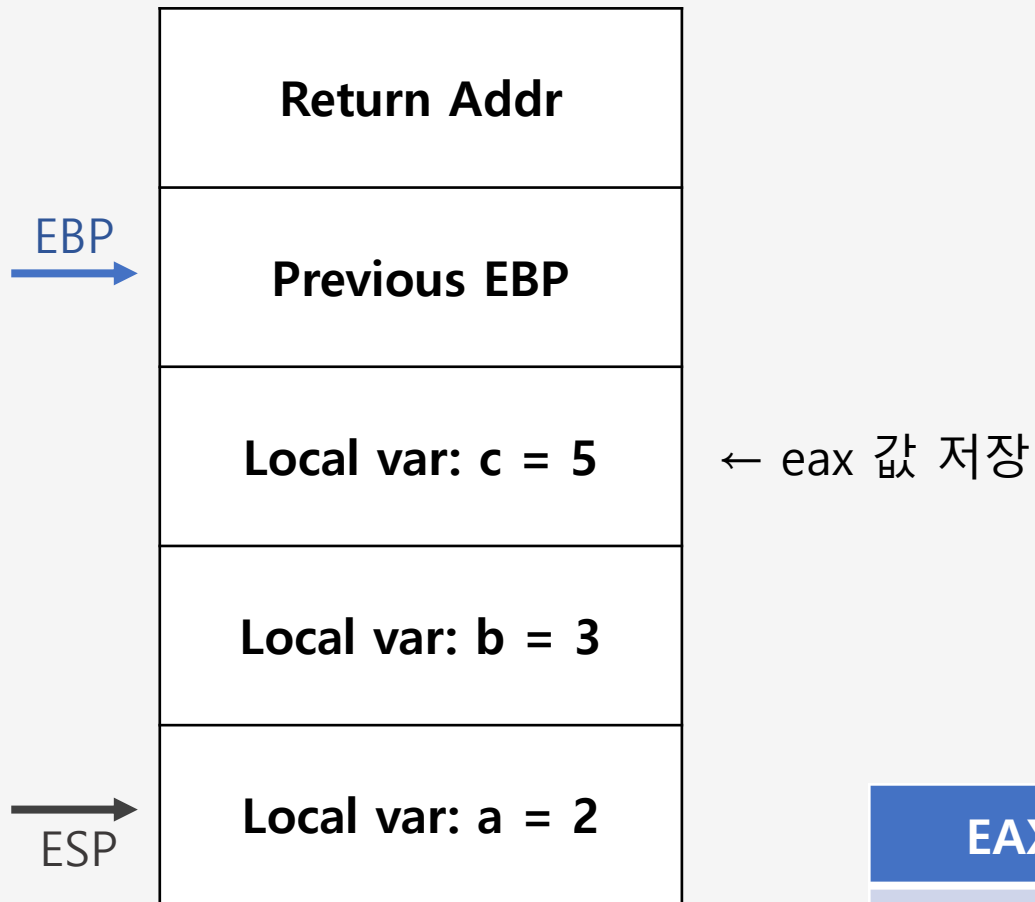
낮은 주소



```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

sum 리턴 → main에서 c = 5 저장

높은 주소



낮은 주소

EAX
5

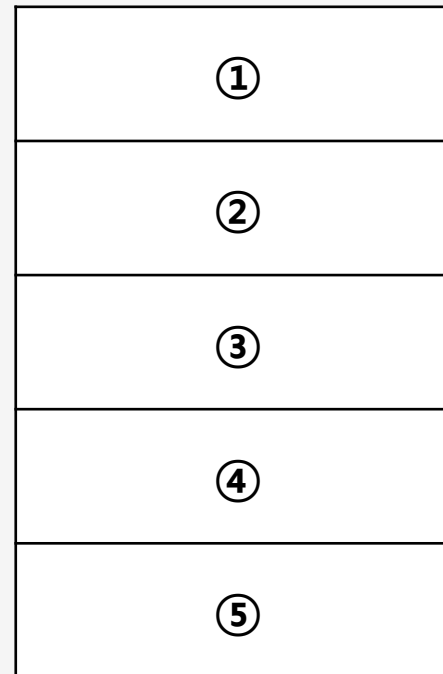
	return addr
ebp →	saved ebp
	local: c
	local: b
esp →	local: a

```
1  #include <stdio.h>
2
3  int sum(int x, int y) {
4      return x + y;
5  }
6
7  int main() {
8      int a = 2, b = 3, c;
9      c = sum(a, b);
10     printf("meow ^._.^");
11 }
```

퀴즈2 mult() 함수 호출 시, mult() 스택 프레임에 push되는 순서는?

```
1     #include<stdio.h>
2
3     int mult(int a, int b){
4         int result = a * b;
5         return result;
6     }
7
8     int main(){
9         int num = mult(3, 5);
10         printf("%d", num);
11         return 0;
12     }
```

높은 주소



낮은 주소

[보기]

Previous EBP

Return Addr

Argv: a

Argv: b

Local: result

퀴즈2 mult() 함수 호출 시, mult() 스택 프레임에 push되는 순서는?

```
1     #include<stdio.h>
2
3     int mult(int a, int b){
4         int result = a * b;
5         return result;
6     }
7
8     int main(){
9         int num = mult(3, 5);
10         printf("%d", num);
11         return 0;
12     }
```

높은 주소

Arg b = 5
Arg a = 3
Return: Addr
Previous EBP
Local: result

낮은 주소

[보기]

Previous EBP

Return Addr

Argv: a

Argv: b

Local: result

과제 소개

복습용 과제

과제 : 강의 내용 정리

- 오늘 다룬 내용을 자유롭게 정리해주세요!
- 요약해도 좋고, 본인이 이해한 방식대로 풀어 써도 좋아요.
- 강의자료 참고, AI 사용, 인터넷 검색 허용.
- 복붙은 금지... (슬퍼할 거임)

제출 방식

- 블로그, 노션, 깃허브 등 자유.
- 공개글로 작성한 후 동아리 노션에 링크를 올려주세요.
- 기한 : 다음주 동아리 시작 전까지 (11/20 18:30까지)
 - 늦제출 : 금요일까지 (11/21 23:59)

요약

- 포너블 개요
- 메모리 4영역
 - 코드, 데이터, 힙, 스택
- 레지스터
 - eip, esp, ebp, eax
- 함수 호출 시 스택 변화
 - 함수 호출 규약, 함수 호출 시/리턴 직전/리턴 후 스택 프레임

2주차 스포일러

- 어셈블리
- gdb
- 스택과 레지스터 추적
- 준비사항
 - VM웨어, 가상머신 설치
 - gcc, gdb 설치
 - <https://www.youtube.com/watch?v=SgfrHKg81Qc>